Implementing Comprehensive Chat Logging in Google NotebookLM with Python

1. Introduction to Python Logging for Conversational AI

The ability to meticulously record the interactions within a conversational AI application is paramount for a multitude of reasons. Effective logging facilitates the debugging process, allowing developers to trace the flow of conversations and pinpoint the origins of errors. It also provides invaluable data for monitoring the application's performance, identifying bottlenecks, and understanding how users are interacting with the system. In the specific context of Google NotebookLM, the need for a persistent and accessible log of chat sessions becomes even more critical. Users like the requester aim to enhance the functionality of NotebookLM by maintaining a running record of their conversations, which can then be reviewed or exported as needed. This necessitates a robust logging mechanism capable of capturing both user inputs and the model's outputs throughout the entirety of a chat session. While the user has successfully managed to extend the conversation history within NotebookLM to a limited extent, the challenge lies in establishing a comprehensive logging system that persists for the entire duration of a session and allows for on-demand retrieval of the complete conversation history. This report delves into the capabilities of Python's built-in logging module and explores strategies for implementing such a system within the constraints and potential extensibility points of Google NotebookLM.

2. Fundamentals of Python's logging Module

Python's logging module offers a sophisticated and highly configurable framework for recording events that occur during the execution of a program [1]. Unlike simple print() statements, which primarily serve for immediate console output, the logging module provides a wealth of features that make it significantly more efficient and informative for tracking application behavior [1]. The use of print() is limited as it requires access to the console and lacks the ability to provide detailed contextual information about the logged event [1]. In contrast, logging allows for the inclusion of structured data such as timestamps, the severity level of the message, and the name of the logger that originated the message, thereby offering a much richer audit trail [1].

The logging module operates based on several core components that work together to manage the logging process [1]. **Loggers** are the primary interface through which application code records events. They manage the logging functionalities and serve as the entry point for emitting log messages [1]. **Handlers** are responsible for directing

log records to specific destinations, which could include the console, files, network streams, or even in-memory buffers [1]. Different types of handlers cater to various output requirements, such as StreamHandler for console output, FileHandler for writing to disk, and MemoryHandler for in-memory buffering [1]. **Formatters** define the structure and layout of log messages as they appear in the output. They allow developers to specify which pieces of information from the log record (like timestamp, level, message) are included and how they are presented [1]. Finally, **Filters** provide an optional mechanism to control which log records are processed by loggers and handlers based on certain criteria, such as the logger name or the severity level [1].

The logging module employs a hierarchical system of **logging levels** to categorize the severity of log messages [1]. These levels, in ascending order of severity, are DEBUG, INFO, WARNING, ERROR, and CRITICAL [1]. The choice of logging level allows developers to filter out less important messages during normal operation and focus on more critical events when necessary. By default, the root logger is set to the WARNING level, meaning that any messages logged at DEBUG or INFO level will be ignored unless the configuration is explicitly changed [1]. To capture all relevant chat inputs and outputs, it will likely be necessary to set the logging level to DEBUG or INFO [8]. The setLevel() method is used to adjust the logging level for both logger objects and handler objects, providing fine-grained control over what gets logged and where [4].

3. Logging Input and Output in Python

To effectively log the conversation within Google NotebookLM, the first step involves identifying the specific points in the Python code where user input is received and where the model's output is generated. Once these locations are identified, the logging module can be used to record these interactions [4]. When a user provides input, a logger method such as logger.info() or logger.debug() can be called to record the input string [4]. It is crucial to include relevant context within the log message to clearly distinguish between user input and model output. For example, a log message for user input could be formatted as "User Input: [actual input]" [8]. Similarly, when the language model generates a response, the output string can be logged using the same logger methods, with a context like "Model Output: [generated text]" [8]. This consistent labeling is essential for making the logs easily understandable and searchable later on.

Structuring the log messages effectively is key to extracting meaningful information from the logs [1]. Python's logging module allows the use of format strings to define the exact information included in each log entry. A common practice is to include a timestamp (%(asctime)s) to record when the event occurred, the log level

(%(levelname)s) to indicate the severity, and the actual log message itself (%(message)s) [1]. For conversational logging, it can be beneficial to include additional context such as a session identifier or a user identifier if these are available within the application's scope [8]. This allows for easier filtering and analysis of logs pertaining to specific sessions or users. An example of a more detailed format string could be %(asctime)s - %(levelname)s - User: %(user_input)s, Model: %(model_output)s. To use custom fields like user_input and model_output in the format string, the extra parameter can be utilized when calling the logger methods [4]. This allows passing a dictionary containing the custom data, which can then be referenced in the format string.

4. Implementing In-Memory Logging with MemoryHandler

Given the user's requirement to keep a running log of conversations within a chat session that can be requested as output, Python's logging.handlers.MemoryHandler offers a particularly relevant solution [1]. The MemoryHandler is a specialized handler class designed to buffer logging records in memory [1]. This means that as the chat session progresses and log messages are generated (capturing user inputs and model outputs), these messages are temporarily stored in the computer's memory. This approach directly addresses the need to maintain a running log that persists throughout the session and can be accessed when the user requests it. The MemoryHandler also has the capability to periodically flush its buffer to a target handler, such as a file handler, but for the immediate requirement of in-session logging and retrieval, this might not be necessary [12].

To implement in-memory logging using MemoryHandler, the first step is to import the logging.handlers module [13]. Next, an instance of the MemoryHandler class needs to be created, and the capacity parameter must be specified [12]. The capacity determines the maximum number of log records that the handler will buffer in memory [12]. The user should choose a capacity that is large enough to accommodate the expected length of typical chat sessions. Optionally, a flushLevel can be set, which defines the severity level at which the buffer will be flushed to a target handler if one is specified [12]. For the purpose of keeping the logs in memory for the entire session, this parameter could be set to a very high level (like logging.CRITICAL) or left at its default, as the primary goal is not to flush prematurely. A target handler can also be optionally specified, which would be another handler (like a FileHandler or SMTPHandler) to which the buffered records are sent when the buffer is full or when a log record reaches the flushLevel [12]. For the user's immediate need, setting a target handler might not be required, as the focus is on accessing the logs from memory within the same session. Finally, the MemoryHandler instance needs to be added to the desired

logger, which could be the root logger or a custom logger specifically created for the chat functionality [4].

The capacity parameter of the MemoryHandler plays a crucial role in managing memory usage. It dictates the maximum number of log messages that will be held in the buffer. Once this capacity is reached, the behavior depends on whether a target handler is set. If a target handler is configured, the buffer will typically be flushed to that handler. If no target handler is set, and the capacity is reached, older log records might be overwritten depending on the specific implementation details of the MemoryHandler. Therefore, it is important to choose a capacity that is sufficiently large to store the entire chat session's log without risking data loss. The flushLevel parameter determines the severity of log messages that will trigger a flush to the target handler. For instance, if the flushLevel is set to logging.ERROR, the buffer will be flushed whenever an error message is logged, in addition to when the capacity is reached. If the primary goal is to keep the log in memory for the duration of the session and only flush (if at all) upon a specific event (like session end), the flushLevel might not be the primary mechanism for controlling log retention.

The following table summarizes the key parameters of the MemoryHandler:

Parameter	Description	Example Value
capacity	Maximum number of log records to buffer in memory.	1000
flushLevel	Severity level at which the buffer should be flushed to the target handler.	logging.ERROR or None
target	Optional handler to which the buffered records will be flushed.	file_handler or None
flushOnClose	Boolean indicating whether to flush the buffer when the handler is closed.	True (default)

5. Retrieving and Presenting the Chat Log

A key challenge when using MemoryHandler is that it does not inherently provide a direct method to retrieve the buffered log records before they are flushed to a target handler [12]. The primary purpose of MemoryHandler is to temporarily hold logs in memory and then pass them on to another handler based on capacity or severity [12]. Therefore, to access the chat log for display to the user, alternative strategies need to be considered.

One approach is to create a **custom handler** that inherits from logging.Handler. Within this custom handler, the emit method can be overridden to store each log record in a list that is maintained within the handler object. This list can then be accessed directly to retrieve the entire history of logged messages. When the user requests the chat log, the application can simply access this list from the custom handler instance and format the log messages for display.

Another strategy involves using the MemoryHandler in conjunction with a **target handler** that is designed to store data in a retrievable format. A particularly suitable target handler for this purpose is the StringIO handler from the io module [9]. The StringIO handler writes log output to an in-memory text buffer, which can then be accessed as a string [9]. In this setup, the MemoryHandler would buffer the log records, and upon reaching its capacity or a specific flush condition, it would flush these records to the StringIO handler. The content of the StringIO buffer, which contains the formatted log messages, can then be retrieved and presented to the user.

Once the log records are retrieved (either from the custom handler's list or the StringIO buffer), the next step is to format the output in a user-friendly manner [1]. This typically involves iterating through the log records (or the lines in the retrieved string) and accessing the relevant attributes of each record, such as the timestamp (asctime), log level (levelname), and the actual message (message) [4]. These attributes can then be combined into a formatted string that clearly presents the conversation history, perhaps in the format " LEVEL: Message". For a chat log, it is essential to clearly differentiate between user inputs and model outputs, which can be achieved by including the context labels ("User:", "Model:") that were used when the messages were initially logged.

Finally, the mechanism for the user to trigger the request for the chat log within Google NotebookLM needs to be considered, along with how the retrieved and formatted log will be presented in the user interface. This would involve understanding the specific capabilities and extension points offered by Google NotebookLM for user

interactions and output display.

6. Exploring Integration with Google NotebookLM and Gemini 1.5

To effectively implement a comprehensive chat logging system, it is essential to understand the architecture and extensibility of Google NotebookLM. Researching any available documentation or information regarding how custom Python code can be integrated or executed within the platform is crucial. The user's success in adding code to the "sources" suggests that this is a viable avenue for introducing custom logging functionality.

The user also inquired about the possibility of using prompt injection techniques or tapping into processes within Gemini 1.5 (the underlying model for NotebookLM) to achieve the desired logging. While prompt injection might theoretically allow for some level of interaction or access to internal functionalities, it is generally not a reliable or secure method for implementing core application features like logging. Prompt injection techniques often rely on exploiting vulnerabilities in the model's prompt parsing and can lead to unpredictable behavior or security risks. Therefore, it is advisable to focus on more robust and officially supported methods of extending NotebookLM's capabilities, such as through the "sources" functionality or any exposed APIs.

Exploring the official documentation for Gemini 1.5 might reveal APIs or functionalities related to session management, logging, or the ability to customize the input and output flow. If Gemini 1.5 offers any hooks or interfaces that allow for intercepting and processing chat messages, these could potentially be leveraged for implementing a logging mechanism. However, it is important to acknowledge that direct access to Google's internal logging mechanisms or low-level processes is likely to be restricted due to security and architectural considerations. The most feasible approach will likely involve implementing the logging functionality within the user's control, possibly through Python code added to the "sources" or by utilizing any officially provided extension points or APIs for Google NotebookLM.

7. Python Code Examples for Chat Log Implementation

The following code examples illustrate how to implement different aspects of chat logging using Python's logging module.

Basic Logging Setup (for initial testing):

Python

```python
import logging

# Configure basic logging to the console
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

logger = logging.getLogger(__name__)

logger.info("Application started.")
user_input = "Hello, NotebookLM!"
logger.info(f"User Input: {user_input}")
model_output = "Hi there! How can I help you?"
logger.info(f"Model Output: {model_output}")
```

Implementing In-Memory Logging with MemoryHandler:

Python

```python
import logging
import logging.handlers
from io import StringIO

# Create a StringIO handler to act as the target for MemoryHandler
log_stream = StringIO()
stream_handler = logging.StreamHandler(log_stream)
formatter = logging.Formatter('%(asctime)s - %(levelname)s - %(message)s')
stream_handler.setFormatter(formatter)

# Create a MemoryHandler with a capacity of 1000 log records
memory_handler = logging.handlers.MemoryHandler(capacity=1000, target=stream_handler, flushLevel=logging.ERROR)

# Get the root logger
```

```
root_logger = logging.getLogger()
root_logger.setLevel(logging.DEBUG)
root_logger.addHandler(memory_handler)

# Log some messages
root_logger.debug("This is a debug message.")
root_logger.info("This is an info message.")
user_input = "What is the capital of France?"
root_logger.info(f"User Input: {user_input}")
model_output = "The capital of France is Paris."
root_logger.info(f"Model Output: {model_output}")

# To retrieve the log content (after it has been flushed to the target StringIO handler,
# which in this example happens on ERROR level or when capacity is reached):
# log_content = log_stream.getvalue()
# print(log_content)

# For immediate retrieval from MemoryHandler (requires flushing):
# memory_handler.flush()
# log_content = log_stream.getvalue()
# print(log_content)
```

Retrieving Logs using a Custom Handler:

Python

```
import logging

class ChatLogHandler(logging.Handler):
    def __init__(self):
        super().__init__()
        self.log_records =

    def emit(self, record):
        self.log_records.append(self.format(record))

# Create a custom handler
chat_handler = ChatLogHandler()
```

```python
formatter = logging.Formatter('%(asctime)s - %(levelname)s - %(message)s')
chat_handler.setFormatter(formatter)

# Get the root logger
root_logger = logging.getLogger()
root_logger.setLevel(logging.INFO)
root_logger.addHandler(chat_handler)

# Log some messages
user_input = "Tell me a joke."
root_logger.info(f"User Input: {user_input}")
model_output = "Why don't scientists trust atoms? Because they make up everything!"
root_logger.info(f"Model Output: {model_output}")

# Function to retrieve the logged messages
def get_chat_log():
    return chat_handler.log_records

# Retrieve and print the chat log
chat_history = get_chat_log()
for log_entry in chat_history:
    print(log_entry)
```

Formatting Log Output:

The formatting of log output can be controlled through the Formatter class. The format string allows specifying the attributes of the LogRecord to include and their presentation. The datefmt parameter can be used to customize the timestamp format [3].

```python
formatter = logging.Formatter('%(asctime)s - %(levelname)s - %(message)s',
datefmt='%Y-%m-%d %H:%M:%S')
```

8. Advanced Logging Considerations for Conversational Systems

For conversational systems, including accurate timestamps in log messages is crucial

for tracking the precise sequence of interactions [1]. The %(asctime)s attribute in the formatter automatically adds a timestamp to each log entry [1]. The format of this timestamp can be further customized using the datefmt parameter of the Formatter [3].

Beyond the basic input and output, consider logging additional metadata that might be relevant to the conversation, such as user IDs, session IDs, or timestamps for each turn [8]. This can provide valuable context for analysis. The extra parameter in logging calls can be used to include such custom data in the log records [4].

When logging user conversations, it is essential to be mindful of security and privacy [8]. Sensitive information, such as personally identifiable information (PII), should be handled with care and appropriate measures should be taken to protect user privacy. This might involve anonymizing or redacting sensitive data before it is logged or displayed.

9. Recommendations and Best Practices

Based on the analysis, the following recommendations and best practices are provided for implementing chat logging in Google NotebookLM:

- **Choose the Right Logging Level:** Set the logging level to DEBUG or INFO to ensure that all chat inputs and outputs are captured [8].
- **Use a Custom Handler:** Implementing a custom handler that stores log records in a list provides the most direct way to access the entire chat history within the current session.
- **Format Log Messages Clearly:** Use a Formatter to include timestamps and clearly distinguish between user input and model output in the log messages.
- **Implement a Mechanism to Trigger Log Retrieval:** Ensure that there is a user-friendly way to request and display the accumulated chat log within the Google NotebookLM interface.
- **Test Thoroughly:** Rigorously test the logging implementation to verify that all interactions are being logged correctly and that the log can be retrieved and displayed as expected.
- **Be Aware of Potential Limitations:** Recognize that direct access to Google's internal logging mechanisms is unlikely. Focus on implementing the logging functionality through the available extension points, such as the "sources" functionality.

10. Conclusion

Comprehensive logging is a fundamental aspect of developing robust and

user-friendly conversational AI applications like Google NotebookLM. By leveraging Python's built-in logging module, and particularly by considering the use of a custom handler to maintain an in-memory record of chat interactions, the user can effectively implement the desired functionality of keeping a running log of conversations that can be requested as output. This capability will not only aid in debugging and analysis but also enhance the user experience by providing transparency and access to their complete conversation history within Google NotebookLM.

Works cited

1. Python Logging Tutorial: How-To, Basic Examples & Best Practices - Sematext, accessed March 20, 2025, https://sematext.com/blog/python-logging/
2. Python Logging Basics - The Ultimate Guide To Logging - Loggly, accessed March 20, 2025, https://www.loggly.com/ultimate-guide/python-logging-basics/
3. Logging in Python - Real Python, accessed March 20, 2025, https://realpython.com/python-logging/
4. logging — Logging facility for Python — Python 3.13.2 documentation, accessed March 20, 2025, https://docs.python.org/3/library/logging.html
5. Logging HOWTO — Python 3.13.2 documentation, accessed March 20, 2025, https://docs.python.org/3/howto/logging.html
6. Logging in Python - GeeksforGeeks, accessed March 20, 2025, https://www.geeksforgeeks.org/logging-in-python/
7. Chapter 15 - Logging — Python 101 1.0 documentation, accessed March 20, 2025, https://python101.pythonlibrary.org/chapter15_logging.html
8. A Comprehensive Guide to Logging in Python | Better Stack Community, accessed March 20, 2025, https://betterstack.com/community/guides/logging/how-to-start-logging-with-python/
9. Python Logging Tutorial - DevDungeon, accessed March 20, 2025, https://www.devdungeon.com/content/python-logging-tutorial
10. Logging Cookbook — Python 3.13.2 documentation, accessed March 20, 2025, https://docs.python.org/3/howto/logging-cookbook.html
11. Complete Python Logging Guide: Best Practices & Implementation - Uptrace, accessed March 20, 2025, https://uptrace.dev/blog/python-logging
12. logging.handlers.MemoryHandler - Red Dove Consultants, accessed March 20, 2025, https://old.red-dove.com/logging/public/logging.handlers.MemoryHandler-class.html
13. logging.handlers — Logging handlers — Python 3.13.2 documentation, accessed March 20, 2025, https://docs.python.org/3/library/logging.handlers.html
14. How to log using MemoryHandler in Python - Stack Overflow, accessed March 20, 2025, https://stackoverflow.com/questions/73726480/how-to-log-using-memoryhandler-in-python

15. io — Core tools for working with streams — Python 3.13.2 documentation, accessed March 20, 2025, https://docs.python.org/3/library/io.html
16. capturing python logging output to a string buffer - GitHub Gist, accessed March 20, 2025, https://gist.github.com/6712491
17. The Ultimate Guide to Python Logging Essentials - ArjanCodes, accessed March 20, 2025, https://arjancodes.com/blog/python-logging-module-implementation-guide/